

Deadlocks in Operating Systems

1. What Is a Deadlock?

A **deadlock** is a situation in an operating system where a set of processes are permanently blocked because each process is waiting for a resource that is held by another process in the same set.

In simple words:

A deadlock happens when processes wait for each other forever, and none of them can continue execution.

Simple Example

Suppose there are two processes:

- P1 holds resource R1 and requests resource R2.
- P2 holds resource R2 and requests resource R1.

Now:

- P1 cannot continue until it gets R2.
- P2 cannot continue until it gets R1.
- But neither process releases its current resource.

So both processes wait forever. This is a deadlock.

Real-Life Example

Imagine two people trying to cross a narrow bridge from opposite sides. Each person waits for the other to move back, but neither moves. Both remain stuck.

This is similar to deadlock in an operating system.

2. Resources in Operating Systems

A **resource** is anything a process needs to execute.

Examples:

- CPU time
- Memory
- Printers
- Files
- Disk drives
- I/O devices
- Semaphores
- Locks
- Database records

Resources can be classified into two types.

2.1 Preemptable Resources

A **preemptable resource** can be taken away from a process without causing serious problems.

Example:

- CPU
- Main memory in some cases

If a process is using the CPU, the operating system can interrupt it and give the CPU to another process.

2.2 Non-Preemptable Resources

A **non-preemptable resource** cannot be taken away safely from a process.

Example:

- Printer
- File lock
- Mutex lock
- Tape drive

If a process is printing a document, taking the printer away in the middle may create incorrect output.

Deadlocks usually occur because of **non-preemptable resources**.

3. Four Necessary Conditions for Deadlock

Deadlock can occur only if all four of the following conditions hold at the same time.

These are also called the **Coffman conditions**.

The four necessary conditions are:

1. Mutual exclusion
2. Hold and wait
3. No preemption
4. Circular wait

If even one of these conditions is removed, deadlock cannot occur.

4. Mutual Exclusion

Meaning

The **mutual exclusion** condition means that at least one resource must be held in a non-shareable mode.

Only one process can use that resource at a time.

If another process requests the same resource, it must wait until the resource is released.

Example

A printer is a non-shareable resource.

Only one process can print at a time. If **P1** is using the printer, then **P2** must wait.

Why It Causes Deadlock

When resources cannot be shared, processes may block while waiting for exclusive access.

Can Mutual Exclusion Be Removed?

For some resources, yes.

For example:

- Read-only files can be shared.
- Multiple processes can read the same file at the same time.

But for resources like printers, locks, and write access to files, mutual exclusion is necessary.

So this condition cannot always be removed.

5. Hold and Wait

Meaning

The **hold and wait** condition means that a process is holding at least one resource and waiting to acquire additional resources that are currently held by other processes.

Example

Suppose:

- **P1** holds **R1** and requests **R2**.
- **P2** holds **R2**.

Here **P1** is holding one resource while waiting for another.

Why It Causes Deadlock

If many processes hold resources and wait for more resources, they may form a chain of waiting.

This chain can eventually become circular, causing deadlock.

How to Prevent Hold and Wait

There are two common methods:

1. A process must request all required resources before execution begins.
2. A process must release all currently held resources before requesting new resources.

Disadvantages

- Low resource utilization.
 - A process may hold resources for a long time even when it does not need them immediately.
 - Starvation may occur because a process may wait forever to get all required resources at once.
-

6. No Preemption

Meaning

The **no preemption** condition means that resources cannot be forcibly taken away from a process.

A resource can be released only voluntarily by the process holding it.

Example

If a process holds a printer, the operating system usually cannot forcibly take the printer away before the printing job is complete.

Why It Causes Deadlock

If a process holds a resource and refuses to release it until it gets another resource, other processes may remain blocked.

How to Prevent No Preemption

If a process holding some resources requests another resource that cannot be immediately allocated, then:

- The operating system can force the process to release its currently held resources.
- The process is restarted later when all needed resources are available.

Disadvantages

- This method works only for resources whose state can be saved and restored.
 - It is not practical for resources like printers or locks in many cases.
-

7. Circular Wait

Meaning

The **circular wait** condition means that there exists a circular chain of processes where each process is waiting for a resource held by the next process in the chain.

Example

Suppose:

- P1 is waiting for a resource held by P2.
- P2 is waiting for a resource held by P3.
- P3 is waiting for a resource held by P1.

This forms a cycle:

```
P1 -> P2 -> P3 -> P1
```

Since each process is waiting for another process in the cycle, none of them can proceed.

How to Prevent Circular Wait

Assign a unique number or priority to every resource type.

A process must request resources only in increasing order of resource numbers.

Example:

If resources are ordered as:

```
R1 < R2 < R3 < R4
```

Then a process that holds **R1** can request **R2**, **R3**, or **R4**.

But a process that holds **R3** cannot request **R1** or **R2**.

This prevents cycles from forming.

Disadvantages

- Resource ordering may be difficult in large systems.
- It may reduce flexibility.
- Processes may request resources earlier than needed, reducing efficiency.

8. Resource Allocation Graph

A **resource allocation graph**, also called a **RAG**, is a graph used to represent the allocation of resources to processes and the requests made by processes.

Components

The graph contains:

- Process nodes
- Resource nodes
- Request edges
- Assignment edges

Process Node

A process is usually represented by a circle.

Example:

(P1)

Resource Node

A resource is usually represented by a rectangle.

Example:

[R1]

If a resource type has multiple instances, dots are shown inside the rectangle.

Request Edge

A request edge goes from a process to a resource.

It means the process is requesting that resource.

Example:

P1 -> R1

Meaning:

P1 is requesting R1.

Assignment Edge

An assignment edge goes from a resource to a process.

It means the resource has been allocated to that process.

Example:

R1 -> P1

Meaning:

R1 is allocated to P1.

Cycle in Resource Allocation Graph

If the graph contains a cycle, deadlock may or may not exist depending on the number of instances of each resource type.

Case 1: One Instance per Resource Type

If each resource type has only one instance:

- A cycle means deadlock definitely exists.

Case 2: Multiple Instances per Resource Type

If resource types have multiple instances:

- A cycle does not necessarily mean deadlock.
 - Deadlock may exist, but it is not guaranteed.
-

9. Deadlock Handling Methods

There are four main ways to handle deadlocks:

1. Deadlock prevention
2. Deadlock avoidance
3. Deadlock detection
4. Deadlock recovery

Another practical approach is to ignore deadlocks, also called the **ostrich algorithm**.

Part A: Deadlock Prevention

10. What Is Deadlock Prevention?

Deadlock prevention means designing the system in such a way that at least one of the four necessary conditions for deadlock can never hold.

Since deadlock requires all four conditions, breaking any one condition prevents deadlock.

Main Idea

Deadlock prevention attacks one of these conditions:

- Mutual exclusion
 - Hold and wait
 - No preemption
 - Circular wait
-

11. Preventing Mutual Exclusion

Mutual exclusion can be prevented by making resources shareable whenever possible.

Example

Instead of giving direct printer access to many processes, the OS can use **spooling**.

In printer spooling:

- Processes write their output to a disk queue.
- The printer daemon prints the jobs one by one.
- Processes do not directly hold the printer.

This reduces the chance of deadlock.

Limitation

Not all resources can be shared.

Examples:

- Mutex locks
- Write access to files
- Printers

So preventing mutual exclusion is not always possible.

12. Preventing Hold and Wait

There are two methods.

Method 1: Request All Resources at Once

A process must request all resources before it starts execution.

If all resources are available, they are allocated.

If not, the process waits and receives nothing.

Example

If a process needs:

- Printer
- File
- Scanner

It must request all three at the beginning.

Advantage

- Deadlock is prevented because a process never holds one resource while waiting for another.

Disadvantages

- Poor resource utilization.
- Processes may hold resources they do not use immediately.

- Starvation is possible.
- The process must know all required resources in advance.

Method 2: Request Resources Only When Holding None

A process can request resources only when it is holding no resources.

If it needs a new resource, it must first release all resources it currently holds.

Advantage

- Prevents hold and wait.

Disadvantage

- May cause repeated release and reacquisition of resources.
 - Can reduce performance.
-

13. Preventing No Preemption

The system can prevent no preemption by forcibly taking resources away from processes in some situations.

Method

If a process holds resources and requests another unavailable resource:

1. The process is forced to release its currently held resources.
2. The released resources are added back to the available pool.
3. The process is restarted later.

Example

Suppose:

- P1 holds R1.
- P1 requests R2.
- R2 is not available.

The OS may take R1 away from P1 and make P1 wait until both R1 and R2 are available.

Limitation

This is useful only for preemptable resources.

It is difficult or impossible for:

- Printers
 - Locks
 - Files being modified
-

14. Preventing Circular Wait

Circular wait can be prevented by imposing a total ordering of resources.

Method

Each resource type is assigned a unique number.

Processes must request resources in increasing order of numbers.

Example

Suppose:

```
R1 = 1  
R2 = 2  
R3 = 3
```

A process can request:

```
R1 -> R2 -> R3
```

But it cannot request:

```
R3 -> R1
```

This prevents circular wait because a cycle would require a process to request a lower-numbered resource after holding a higher-numbered resource.

Advantage

- Simple and commonly used in locking systems.

Disadvantages

- Choosing the correct resource ordering can be hard.
- Processes may acquire resources before they are needed.
- System flexibility is reduced.

15. Advantages and Disadvantages of Deadlock Prevention

Advantages

- Guarantees that deadlock will not occur.
- Conceptually simple.
- Useful in systems where deadlock must never happen.

Disadvantages

- Can reduce resource utilization.
- Can reduce system throughput.
- May cause starvation.
- Often too restrictive.
- Requires strong assumptions about resource needs.

Part B: Deadlock Avoidance

16. What Is Deadlock Avoidance?

Deadlock avoidance means dynamically checking each resource request before granting it, to make sure the system will remain in a safe state.

Unlike prevention, avoidance does not break one of the four deadlock conditions permanently.

Instead, it carefully decides whether a resource request should be granted.

Main Idea

Before allocating a resource, the operating system asks:

If I grant this request, will the system still be safe?

If yes, the request is granted.

If no, the process must wait.

17. Safe State

A system is in a **safe state** if there exists at least one sequence of process execution such that all processes can finish without causing deadlock.

This sequence is called a **safe sequence**.

Safe Sequence

A **safe sequence** is an order of processes in which every process can get its required resources, finish execution, and release its resources for the next process.

Example:

<P1, P3, P2>

This means:

1. **P1** can finish with currently available resources.

2. After P1 finishes, it releases its resources.
3. P3 can then finish.
4. After P3 finishes, it releases its resources.
5. P2 can finally finish.

If such a sequence exists, the system is safe.

18. Unsafe State

A system is in an **unsafe state** if there is no safe sequence.

Important point:

Unsafe state does not always mean deadlock.

An unsafe state means the system may enter deadlock in the future depending on future requests.

Relationship

```
Deadlock state -> Unsafe state
Unsafe state -> May or may not be deadlock
Safe state -> No deadlock
```

So:

- Every deadlock state is unsafe.
 - Not every unsafe state is a deadlock state.
-

19. Requirements for Deadlock Avoidance

Deadlock avoidance requires advance information about processes.

The OS must know:

- Maximum number of resources each process may request.
- Currently allocated resources.
- Currently available resources.
- Remaining need of each process.

This is why avoidance is harder to implement than prevention.

Part C: Banker's Algorithm

20. What Is the Banker's Algorithm?

The **Banker's algorithm** is a deadlock avoidance algorithm used when there are multiple instances of each resource type.

It was proposed by Edsger Dijkstra.

It is called the Banker's algorithm because it is similar to a banker giving loans.

A banker gives a loan only if the bank can still satisfy the possible needs of all customers. Similarly, the operating system allocates resources only if the system remains in a safe state.

21. Main Data Structures in Banker's Algorithm

Assume:

- n = number of processes
- m = number of resource types

The Banker's algorithm uses the following data structures.

21.1 Available

Available is a vector of length m .

It shows the number of available instances of each resource type.

Example:

```
Available = [3, 3, 2]
```

This means:

- 3 instances of resource A are available.
- 3 instances of resource B are available.
- 2 instances of resource C are available.

21.2 Max

Max is an $n \times m$ matrix.

It shows the maximum demand of each process for each resource type.

Example:

```
Max[P1] = [7, 5, 3]
```

This means process **P1** may need at most:

- 7 instances of A

- 5 instances of B
- 3 instances of C

21.3 Allocation

Allocation is an $n \times m$ matrix.

It shows the resources currently allocated to each process.

Example:

```
Allocation[P1] = [0, 1, 0]
```

This means process **P1** currently holds:

- 0 instances of A
- 1 instance of B
- 0 instances of C

21.4 Need

Need is an $n \times m$ matrix.

It shows the remaining resources each process may still request.

Formula:

```
Need = Max - Allocation
```

Example:

```
Max[P1]          = [7, 5, 3]
Allocation[P1]   = [0, 1, 0]
Need[P1]         = [7, 4, 3]
```

22. Safety Algorithm

The safety algorithm checks whether the system is in a safe state.

Steps

1. Initialize:

```
Work = Available
```

```
Finish[i] = false for all processes
```

2. Find a process P_i such that:

```
Finish[i] == false  
Need[i] <= Work
```

This means the process has not finished and its remaining need can be satisfied using currently available resources.

3. Pretend that P_i finishes.

When P_i finishes, it releases its allocated resources:

```
Work = Work + Allocation[i]  
Finish[i] = true
```

4. Repeat steps 2 and 3 until no more processes can be found.

5. If all $Finish[i]$ values are true, the system is safe.

If at least one $Finish[i]$ is false, the system is unsafe.

23. Resource Request Algorithm

This algorithm decides whether a process request should be granted.

Suppose process P_i requests resources:

```
Request[i]
```

Step 1: Check Request Is Within Need

```
Request[i] <= Need[i]
```

If false, the process has requested more than its declared maximum need. This is an error.

Step 2: Check Request Is Within Available

```
Request[i] <= Available
```

If false, resources are not currently available, so the process must wait.

Step 3: Pretend to Allocate

Temporarily allocate the requested resources:

```
Available = Available - Request[i]
Allocation[i] = Allocation[i] + Request[i]
Need[i] = Need[i] - Request[i]
```

Step 4: Run Safety Algorithm

If the new state is safe:

- Grant the request permanently.

If the new state is unsafe:

- Roll back the temporary allocation.
- Make the process wait.

24. Banker's Algorithm Example

Consider 5 processes and 3 resource types:

```
Processes: P0, P1, P2, P3, P4
Resources: A, B, C
Available = [3, 3, 2]
```

Allocation Matrix

	A	B	C
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

Max Matrix

	A	B	C
P0	7	5	3
P1	3	2	2
P2	9	0	2

P3	2 2 2
P4	4 3 3

Need Matrix

Using:

$$\text{Need} = \text{Max} - \text{Allocation}$$

We get:

	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

Finding Safe Sequence

Initial:

$$\text{Work} = \text{Available} = [3, 3, 2]$$

Check each process:

- P0 needs [7, 4, 3], which is more than Work, so it cannot finish now.
- P1 needs [1, 2, 2], which is less than or equal to Work, so P1 can finish.

After P1 finishes:

$$\begin{aligned} \text{Work} &= \text{Work} + \text{Allocation}[P1] \\ \text{Work} &= [3, 3, 2] + [2, 0, 0] \\ \text{Work} &= [5, 3, 2] \end{aligned}$$

Now:

- P3 needs [0, 1, 1], so P3 can finish.

After P3 finishes:

$$\text{Work} = [5, 3, 2] + [2, 1, 1]$$

```
Work = [7, 4, 3]
```

Now:

- P0 needs [7, 4, 3], so P0 can finish.

After P0 finishes:

```
Work = [7, 4, 3] + [0, 1, 0]
Work = [7, 5, 3]
```

Now:

- P2 needs [6, 0, 0], so P2 can finish.

After P2 finishes:

```
Work = [7, 5, 3] + [3, 0, 2]
Work = [10, 5, 5]
```

Now:

- P4 needs [4, 3, 1], so P4 can finish.

After P4 finishes:

```
Work = [10, 5, 5] + [0, 0, 2]
Work = [10, 5, 7]
```

All processes can finish.

So the system is in a safe state.

Safe sequence:

```
<P1, P3, P0, P2, P4>
```

25. Advantages of Banker's Algorithm

- Avoids deadlock.
- Supports multiple instances of resources.
- Ensures the system stays in a safe state.
- Useful for understanding deadlock avoidance theoretically.

26. Disadvantages of Banker's Algorithm

- Requires advance knowledge of maximum resource needs.
 - Number of processes must usually remain fixed.
 - Number of resources must usually remain fixed.
 - Processes may not know their maximum needs in advance.
 - It can be expensive to run safety checks frequently.
 - It may reduce resource utilization because some safe-looking requests are delayed if they lead to unsafe states.
 - It is rarely used directly in general-purpose operating systems.
-

Part D: Deadlock Detection

27. What Is Deadlock Detection?

Deadlock detection means allowing deadlocks to occur, then periodically checking whether a deadlock has happened.

If a deadlock is detected, the system uses recovery techniques to break it.

Main Idea

Instead of preventing or avoiding deadlock, the OS says:

Let processes request resources normally. If deadlock occurs, detect it and recover.

28. Detection with Single Instance of Each Resource Type

If each resource type has only one instance, deadlock can be detected using a **wait-for graph**.

Wait-For Graph

A wait-for graph contains only process nodes.

An edge:

$P_i \rightarrow P_j$

means:

P_i is waiting for a resource held by P_j .

Deadlock Rule

If the wait-for graph contains a cycle, deadlock exists.

Example

```
P1 -> P2 -> P3 -> P1
```

This cycle means:

- **P1** waits for **P2**.
- **P2** waits for **P3**.
- **P3** waits for **P1**.

So **P1**, **P2**, and **P3** are deadlocked.

29. Detection with Multiple Instances of Resource Types

When resource types have multiple instances, a detection algorithm similar to the Banker's safety algorithm is used.

Data Structures

The algorithm uses:

Available

A vector showing available instances of each resource type.

Allocation

A matrix showing currently allocated resources for each process.

Request

A matrix showing the current outstanding request of each process.

$\text{Request}[i][j] = k$ means process **P_i** is currently requesting **k** instances of resource type **R_j**.

30. Deadlock Detection Algorithm

Steps

1. Initialize:

```
Work = Available
```

For each process **P_i**:

```
if Allocation[i] != 0:
    Finish[i] = false
else:
    Finish[i] = true
```

Processes with no allocated resources cannot be part of a deadlock, so they are marked true.

2. Find a process P_i such that:

```
Finish[i] == false
Request[i] <= Work
```

3. Pretend P_i finishes and releases its resources:

```
Work = Work + Allocation[i]
Finish[i] = true
```

4. Repeat until no such process can be found.

5. If any process has:

```
Finish[i] == false
```

then that process is deadlocked.

31. How Often Should Deadlock Detection Run?

Deadlock detection can be run:

- Whenever a resource request cannot be granted.
- At fixed time intervals.
- When CPU utilization drops below a threshold.
- When system performance becomes poor.

Frequent Detection

Advantages:

- Deadlocks are found quickly.
- Recovery is easier because fewer processes may be involved.

Disadvantages:

- High overhead.

Infrequent Detection

Advantages:

- Lower overhead.

Disadvantages:

- Deadlocks may remain for a long time.
 - Many processes may get involved, making recovery harder.
-

Part E: Deadlock Recovery

32. What Is Deadlock Recovery?

Deadlock recovery means taking action to break a detected deadlock.

Once a deadlock is detected, the system must recover by:

1. Terminating processes, or
 2. Preempting resources.
-

33. Recovery by Process Termination

The OS can recover by killing one or more processes involved in the deadlock.

There are two methods.

Method 1: Abort All Deadlocked Processes

All processes involved in the deadlock are terminated.

Advantage

- Simple.
- Deadlock is removed immediately.

Disadvantages

- Very costly.
- Work done by all terminated processes is lost.
- May leave files or data in an inconsistent state.

Method 2: Abort One Process at a Time

The OS terminates one deadlocked process at a time until the deadlock cycle is broken.

Advantage

- Less work is lost compared to killing all processes.

Disadvantages

- More overhead.
 - Deadlock detection must be run again after each termination.
 - Choosing the right process to terminate can be difficult.
-

34. Choosing a Victim Process

When terminating a process or preempting its resources, the OS must choose a **victim**.

Factors used to choose a victim:

- Process priority
- How long the process has executed
- How much work remains
- Number and type of resources held
- Number of resources still needed
- Whether the process is interactive or batch
- Cost of rollback
- Importance of the process

Usually, the OS tries to minimize total cost.

35. Recovery by Resource Preemption

The OS may take resources away from one or more processes and give them to other processes.

Steps

1. Select a victim process.
2. Preempt some resources from it.
3. Roll the victim process back to a safe state.
4. Restart the process later.

Example

If **P1**, **P2**, and **P3** are deadlocked, the OS may take a resource from **P3** and give it to **P1**.

Then **P1** may finish and release its resources, breaking the deadlock.

36. Rollback

Rollback means returning a process to an earlier safe state.

This requires the system to save checkpoints during execution.

Example

If a process was executing a transaction, the system can roll it back to the state before the transaction began.

Types of Rollback

Total Rollback

The process is restarted from the beginning.

Partial Rollback

The process is rolled back only as far as necessary to break the deadlock.

Disadvantages

- Checkpointing adds overhead.
- Rollback may be difficult for I/O operations.
- Some operations cannot be undone easily.

37. Starvation During Recovery

Starvation can occur if the same process is repeatedly chosen as the victim.

To avoid starvation:

- Track how many times a process has been selected as victim.
- Increase its priority after rollback.
- Avoid selecting the same process repeatedly.

Part F: Comparison Tables

38. Prevention vs Avoidance

Point	Deadlock Prevention	Deadlock Avoidance
Meaning	Ensures at least one deadlock condition never holds	Grants requests only if system remains safe
Main idea	Restrict process/resource behavior	Dynamically examine each request
Prior knowledge needed	Usually less	Maximum resource needs must be known
Resource utilization	Often low	Better than prevention
Flexibility	Low	Higher
Runtime checking	Less	More
Example	Resource ordering	Banker's algorithm

39. Avoidance vs Detection

Point	Deadlock Avoidance	Deadlock Detection
Deadlock allowed?	No	Yes
Action timing	Before allocation	After deadlock may occur
Main concept	Safe state	Detect cycle or blocked set
Recovery needed?	No	Yes
Overhead	Safety check before allocation	Periodic detection and recovery
Example	Banker's algorithm	Wait-for graph

40. Prevention vs Detection and Recovery

Point	Prevention	Detection and Recovery
Strategy	Stop deadlock before it can occur	Allow deadlock, then fix it
Resource utilization	Lower	Higher before deadlock occurs
Complexity	Policy restrictions	Detection and recovery algorithms
Suitable for	Critical systems	Systems where deadlock is rare
Cost	Continuous restrictions	Recovery cost when deadlock occurs

Part G: Important Exam Points

41. Frequently Asked Questions

Q1. What are the four necessary conditions for deadlock?

The four necessary conditions are:

1. Mutual exclusion
2. Hold and wait
3. No preemption
4. Circular wait

Deadlock can occur only if all four conditions hold simultaneously.

Q2. Are the four conditions sufficient for deadlock?

In general, the four conditions are necessary for deadlock. In systems with a single instance of each resource type, a cycle in the resource allocation graph is also sufficient for deadlock.

With multiple instances, a cycle may indicate the possibility of deadlock, but it does not always guarantee deadlock.

Q3. What is the difference between safe, unsafe, and deadlocked states?

- Safe state: There exists a safe sequence, so deadlock will not occur.
- Unsafe state: No safe sequence exists, but deadlock may or may not have occurred.
- Deadlocked state: Processes are permanently blocked.

Q4. Why is Banker's algorithm not commonly used in real operating systems?

Because it requires knowing each process's maximum resource needs in advance, which is often unrealistic. It also adds runtime overhead and assumes a relatively fixed set of processes and resources.

Q5. What is the difference between prevention and avoidance?

Prevention ensures deadlock cannot occur by breaking one of the necessary conditions. Avoidance allows the conditions to exist but carefully checks each allocation to keep the system in a safe state.

Q6. What is the difference between avoidance and detection?

Avoidance prevents the system from entering an unsafe state. Detection allows deadlocks to occur and then identifies them so that recovery can be performed.

Q7. What is a safe sequence?

A safe sequence is an order of process execution in which every process can complete using currently available resources plus resources released by previously completed processes.

Q8. What is the wait-for graph?

A wait-for graph is a graph used for deadlock detection when each resource type has only one instance. It contains only process nodes. A cycle in the wait-for graph means deadlock exists.

Q9. What is the ostrich algorithm?

The ostrich algorithm means ignoring the deadlock problem. Some systems use this approach if deadlocks are very rare and the cost of prevention, avoidance, or detection is too high.

Q10. What are common recovery techniques?

Common recovery techniques are:

- Process termination
- Resource preemption
- Rollback

42. Short Notes for Quick Revision

Deadlock

A condition where a set of processes wait forever for resources held by each other.

Four Necessary Conditions

Deadlock requires mutual exclusion, hold and wait, no preemption, and circular wait.

Prevention

Break at least one necessary condition so deadlock cannot occur.

Avoidance

Grant a resource request only if the resulting state is safe.

Banker's Algorithm

A deadlock avoidance algorithm that uses **Available**, **Max**, **Allocation**, and **Need** to check whether resource allocation keeps the system safe.

Detection

Allow deadlock to happen, then detect it using wait-for graphs or detection algorithms.

Recovery

Break deadlock by killing processes, preempting resources, or rolling processes back.

43. One-Line Interview Answers

- **Deadlock:** A situation where processes wait indefinitely for resources held by one another.
 - **Mutual exclusion:** Only one process can use a resource at a time.
 - **Hold and wait:** A process holds resources while waiting for more.
 - **No preemption:** Resources cannot be forcibly taken away.
 - **Circular wait:** Processes form a cycle of waiting.
 - **Deadlock prevention:** Break one of the four necessary conditions.
 - **Deadlock avoidance:** Allocate resources only if the system remains in a safe state.
 - **Safe state:** A state where all processes can finish in some order.
 - **Unsafe state:** A state with no safe sequence; deadlock may occur.
 - **Banker's algorithm:** A resource allocation algorithm that avoids deadlock by checking safety before granting requests.
 - **Deadlock detection:** Find whether deadlock has occurred.
 - **Deadlock recovery:** Break deadlock after detection.
-

44. Common Mistakes to Avoid

- Do not say that an unsafe state is always a deadlock. It is not.
 - Do not say that a cycle always means deadlock when resources have multiple instances.
 - Do not confuse deadlock prevention with deadlock avoidance.
 - Do not forget that Banker's algorithm needs maximum resource demand in advance.
 - Do not write only the names of the four conditions in exams; explain each condition with examples.
 - Do not confuse starvation with deadlock.
-

45. Deadlock vs Starvation

Point	Deadlock	Starvation
Meaning	Processes wait forever for each other	A process waits indefinitely because resources are repeatedly given to others
Cause	Circular waiting	Unfair scheduling or allocation
Processes involved	Usually a set of processes	Usually one or more neglected processes
Resource movement	No process in the cycle progresses	Other processes may continue progressing
Solution	Prevention, avoidance, detection, recovery	Aging, fair scheduling, priority adjustment

Example of Starvation

In priority scheduling, a low-priority process may wait forever if high-priority processes keep arriving.

This is starvation, not deadlock, because other processes are still making progress.

46. Final Summary

Deadlock is one of the most important concepts in operating systems. It occurs when processes are permanently blocked because each process is waiting for a resource held by another process. Deadlock requires four necessary conditions: mutual exclusion, hold and wait, no preemption, and circular wait.

Operating systems handle deadlocks using prevention, avoidance, detection, and recovery. Prevention breaks one of the necessary conditions. Avoidance checks each allocation carefully and keeps the system in a safe state. The Banker's algorithm is the classic avoidance algorithm. Detection allows deadlocks to occur and then finds them using graphs or algorithms. Recovery breaks deadlocks by terminating processes, preempting resources, or rolling processes back.

For placements, remember the difference between prevention, avoidance, detection, and recovery clearly, because interviewers often test these comparisons.